

Why Runner: Programming Online Judge

Juan Israel dos Anjos

April 2024

Federal Institute of Education, Science and Technology of Minas Gerais (IFMG) Rua Afonso Sardinha, 90, Bairro Minas Talco – Ouro Branco/MG – CEP: 36494-018 – Brazil

Abstract

Existing online judges primarily focus on correctness-based evaluation, often neglecting pedagogical feedback and real-time contest dynamics essential for educational contexts. This paper introduces WhyRunner: Programming Online Judge, a modular and secure system that integrates containerized code execution, real-time contest management, and AI-powered feedback. Tested in classroom environments with over 50 submissions, WhyRunner demonstrates improved student engagement and educational results through detailed, automated feedback and flexible contest configurations.

Introduction

Programming competitions brings up algorithmic thinking and problem-solving skills. However, many online judges lack features optimized for education, such as flexible contest configurations, intelligent feedback, and secure multi-language support. Runner addresses these gaps by providing a modular, AI-enhanced judge designed for both contests and open practice.

This paper presents the system's architecture, development methodology, implementation, and evaluation in classroom environments.

Related Work

Ataux

The objective of Ataux is the creation of programming problems lists. A administrator can create lists and the class will submit solutions to be reviewed, generating a sheet with the results.

The system offers different kinds of users (admins, collaborators and members), creating problems lists, viewing the solutions and the result sheets. Furthermore, the members can filter questions by difficulty and type, register themselves in new teams and solve questions through Codeforces.

The architecture of Ataux is based on a RESTful Client-Server model. The front-end is developed using React (JavaScript), with Redux for state management and Bootstrap for UI styling. The back-end is built with NestJS (TypeScript), utilizing PostgreSQL as the database, accessed through TypeORM. The system is deployed in a production environment on Heroku.

Ataux supports multiple programming languages to allow flexibility in solving problems, although the range is limited compared to larger competitive platforms.

Some limitations of the system include the restricted number of supported languages, less flexibility in problem creation formats, and basic evaluation feedback focused only on correctness.

To assess the system's usability, the Post-Study System Usability Questionnaire (PSSUQ) methodology was applied. The questionnaire was completed by a group of 71 students and 1 teaching assistant, who used the system over a period of two weeks during their programming activities.

GOJ

The objective of GOJ is to store and judge problems from previous editions of the UnB Programming Marathons, while also providing support tools for beginners.

Among its main features are:

- Storage and automatic judging of problems from past events.
- Availability of tutorials designed to assist newcomers in programming competitions.
- Automated judging of submissions.
- Organization of problems by event for easier navigation.
- Administration tools through the Gamma Judge Admin system.

The architecture of GOJ is based on a micro services model and Web APIs. It leverages various AWS services, including SQS (Simple Queue Service), SNS (Simple Notification Service), and S3 (Simple Storage Service), to enhance scalability and reliability. The back-end is developed using .NET, with MongoDB as its primary database. The front-end is built in React, with additional support for rendering HTML and LaTeX elements via React Latex. Submission judging is performed through Bash scripts, ensuring lightweight and efficient code evaluation.

Run Codes

Run Codes is an online judge platform developed by Brazilian universities, designed to create and manage programming contests and activities. It allows teachers to create problem lists and competitions, automates the evaluation of code submissions, and generates reports for performance analysis.

It supports a wide variety of programming languages and integrates with virtual learning environments. However, Run Codes' complexity and external dependencies can make it less flexible for fully customized learning activities or research projects.

BOCA

BOCA (BOCA Online Contest Administrator) is an open-source system widely used in Brazilian programming competitions like the ACM ICPC regional contests. BOCA allows managing contests, teams, problems, and automated judging.

The system is built mainly with PHP and relies on a LAMP stack (Linux, Apache, MySQL, PHP). While robust for traditional contests, its interface and extensibility are more limited compared to newer platforms.

Comparative Analysis

System	AI Feedback	Multi-Language	Real-time Features	Problems Database
Ataux	No	Limited	No	?
GOJ	No	Moderate	Limited	?
Run Codes	No	Extensive	Yes	?
BOCA	No	Moderate	Yes	?
WhyRunner	Yes	Extensive	Yes	?

Methodology

The system development followed an iterative and incremental methodology. This means the system was built step by step, adding features and improvements after each test and evaluation.

Requirements analysis: The problems of existing judges were studied, such as lack of flexibility, no AI feedback, and no real-time features. From this, the necessary system features were defined.

Modular and scalable design: The system was designed to be easy to maintain and expand. Docker containers are used to run user code safely and support many programming languages.

Secure and efficient execution: The system applies limits on CPU, memory, and execution time to prevent misuse. It is optimized to run many submissions concurrently without performance loss.

AI feedback integration: The system provides artificial intelligence feedback to explain errors and suggest improvements, instead of only reporting “wrong answer.”

The system was validated through tests in classrooms and real contests in a discord server. These tests demonstrated good stability and usefulness for both students and teachers. After each test, improvements were made. This approach helped build a better system for programming education.

System Design

Architecture

The high-level architecture includes:

Front-end: Next.js (React-based), interfacing via REST API. Back-end: NestJS for contest logic and queue management. Judge Runners: Docker containers for each supported language (C++, Rust, Java). Queue System: BullMQ + Redis for job prioritization. Database: PostgreSQL managed via Prisma ORM. AI Feedback Service: OpenAI API integration for intelligent feedback.

Diagram:

User → Front-end (Next.js) → Back-end (Bun) → Queue (BullMQ + Redis) → Judge Runner (Docker) → Result → Feedback → Database (PostgreSQL)

Technologies

Next.js: SSR and SSG for front-end. Bun: Modular back-end framework. PostgreSQL: Relational data storage. Prisma: Type-safe ORM. Docker: Secure, isolated code execution. BullMQ: Redis-based job queue. OpenAI API: AI-driven code feedback.

Implementation

The system ensures secure and efficient code execution through Dockerized judge runners:

1. Receive and store submission.
2. Add to queue with dynamic priority.
3. Execute code inside a Docker container.
4. Compare output against test cases.
5. Update submission status and provide AI feedback.

Resource constraints (CPU, memory, execution time) are enforced via Docker configurations. AI feedback enhances educational value by analyzing failed submissions and suggesting improvements.

Evaluation

System Test at Class

A classroom contest simulated real-world conditions:

6 problems of varying difficulty. 23 students participating. 50+ submissions in C++ and Python. Real-time scoreboard and problem announcements.

The system operated reliably without crashes or major delays.

Results

Students efficiently submitted solutions. AI feedback was especially beneficial for beginners, clarifying common mistakes like input parsing errors. Professors valued the modular contest setup and real-time features.

Discussion

Strengths

Modular, scalable architecture. Secure multi-language code execution. AI-driven, educationally focused feedback.

Limitations

Dependence on external AI API. Initial setup complexity due to containerization.

Future Work

Expand supported languages (e.g., Java, Kotlin). Integrate plagiarism detection. Enhance AI feedback with code quality metrics. Implement advanced test case generators.

Conclusion

Runner: Programming Marathon Online Judge successfully delivers a modern, secure, and educational environment for programming contests. Through containerization, asynchronous job handling, and AI feedback, it supports both competition and learning. Its modular design and extensibility make it a valuable platform for future educational applications and research.

References